SQL indeksləri və sütuna indeks verməyin təsiri

İndeks müəyyən axtarışları, birləşmələri və sıralamaları daha az məlumat oxumaqla yerinə yetirmək üçün saxlanan əlavə məlumat strukturudur. Onun faydası konkret sorğudan asılıdır. İndeks sütunun dəyərini dəyişmir və hər sorğunu avtomatik sürətləndirmir. Oxuma qazancının qarşılığında yazma, disk və yaddaş xərci yaranır.

Bu dərs `DatabaseİndexingHTML`, `GIST&GIN&BRIN_IndexHTML`, `files/05_sql.pdf`, `DB_Arxitektura.pdf` və SQL-in daxili işləməsi materiallarını dəqiqləşdirir. SQL Server və PostgreSQL bölmələri açıq ayrılır. Əvvəlki materiallardakı sabit sürət rəqəmləri real ölçmə kimi qəbul edilmir.

Mündəricat

SQL indeksləri və sütuna indeks verməyin təsiri **1**

1 Əvvəlcə cədvəl sətir səhifə və açar . . . 2

2 B plus tree necə axtarır . . . 2

3 SQL Server clustered və nonclustered . . . 2

4 Sütuna indeks verəndə nə dəyişir . . . 3

5 Composite indeksdə sıra niyə vacibdir . . . 3

6 Unique filtered və aşağı cardinality . . . 4

7 SARGability və funksiyanın təsiri . . . 4

8 PostgreSQL indeksləri ayrıca dünyadır . . . 5

9 Planı necə qiymətləndirməli . . . 6

10 Müsahibə məşqi . . . 6

11 Rəsmi mənbələr . . . 6

1 Əvvəlcə cədvəl sətir səhifə və açar

Cədvəl eyni sütun quruluşlu məlumatlar toplusudur. **Sətir** bir qeyddir. **Sütun** həmin qeydin bir xassəsidir. **Key**, açar, indeksdə axtarışı və sıralamanı müəyyən edən dəyər və ya dəyərlərdir. **Page**, səhifə, baza mühərrikinin məlumat saxlayıb oxuduğu blokdur. SQL Server-in adi rowstore məlumat səhifələri 8 KB ölçüdədir. Rowstore bir sətrin sütunlarını sətir əsaslı quruluşda saxlayır.

Bir səhifədə çoxlu sətir yerləşə bilər. Ona görə 1 milyon sətrə baxmaq mütləq 1 milyon disk oxunuşu demək deyil. **Buffer pool** baza səhifələrinin RAM-da saxlanan keşidir. RAM işləyən proqramların sürətli, dəyişkən yaddaşıdır. **Logical read** səhifəyə buffer üzərindən müraciətdir. **Physical read** məlumatı saxlama qurğusundan gətirmək ehtiyacıdır. Keşdə olan səhifə üçün yeni disk oxunuşu tələb olunmaya bilər.

İndeksin kitab kataloquna bənzətməsi axtarış ideyasını izah edir, amma texniki zəmanət vermir. Bir kitabın bir neçə nüsxəsini götürmək və kitabın çox hissəsini oxumaq fərqli xərclərdir; bazada da uyğun nəticə sayı və qaytarılan sütunlar vacibdir.

2 B plus tree necə axtarır

B+ tree balanslı, çoxşaxəli axtarış ağacıdır. **Root** başlanğıc qovşaq, **branch** aralıq qovşaq, **leaf** son səviyyədir. Aralıq səhifələr açar aralıqlarını göstərir. Axtarış hər səviyyədə uyğun aralığa keçir və yarpaq səviyyəsində dəyəri tapır. Balanslı olması yarpaqlara yolun təxminən eyni dərinlikdə qalmasını bildirir.

Məntiqi sxem

```
Root:          [1..5000] [5001..10000]
                  |            |
Branch:      [1..2500] ...  [7501..10000]
                  |
Leaf:       [101, 102, 103, 104, ...]
```

Bu sxem iki uşaqlı binary tree təsviri deyil. Real səhifədə çoxlu açar yerləşir, buna **fan-out**, şaxələnmə sayı deyilir. `O(log n)` axtarışın məlumat artdıqca yavaş böyüyən nəzəri xərcini ifadə edir. `O` böyümə sürətinin işarəsidir, millisanə ölçüsü deyil. Daha dəqiq baxışda ağacın şaxələnməsi və qaytarılan K sətirin xərci də var: yolu tapmaq və uyğun nəticələri oxumaq ayrı işdir.

“1 milyon sətir üçün log2 hesabladım, deməli dəqiq 20 disk oxunuşudur” düzgün deyil. Ağac binary deyil, səhifələr keşdə ola bilər, əlavə lookup lazım gələ bilər və nəticə çox sətrdən ibarət ola bilər. İndeksdə aralıq axtarışı uyğun başlanğıcı tapıb yarpaq səhifələri boyunca davam edə bilər.

3 SQL Server clustered və nonclustered

Clustered rowstore index-in yarpaq səviyyəsi cədvəlin məlumat sətirləridir. Buna görə cədvəldə bu strukturdan bir dənə ola bilər. Açar sırasına uyğun məntiqi quruluş verir; bütün səhifələrin diskdə fasiləsiz fiziki qonşu qalmasına zəmanət vermir. SQL nəticəsinin sırası üçün yenə `ORDER BY` yazılır.

Heap SQL Server-də clustered index-i olmayan cədvəldir. Heap “bütün SQL sistemlərində bütün cədvəllər həmişə belə saxlanır” demək deyil. **Nonclustered index** ayrıca quruluşdur: açarları və sətri tapmaq üçün locator saxlayır. Locator heap-də row identifier, clustered cədvəldə isə clustered açar əsasında işləyir. Locator sətrə çatmaq üçün göstəricidir.

Key lookup nonclustered indeksdən tapılan clustered açarla əsas məlumat sətrinə əlavə gedişdir. **RID lookup** heap sətrinə əlavə gedişdir. Bir nəticə üçün ucuz olan lookup minlərlə nəticə üçün baha ola bilər. Clustered açar genişdirsə nonclustered indekslərin saxladığı locator xərci də arta bilər.

Primary key sətrlərin unikal və NULL olmayan identifikatorunu tələb edən məntiqi constraint-dir. Constraint məlumat üçün məcburi qaydadır. SQL Server çox vaxt ona dəstək üçün indeks yaradır, amma primary key mütləq clustered olmalıdır demək deyil. Cədvəlin ayrıca clustered açarı, primary key-in isə nonclustered indeksi ola bilər.

4 Sütuna indeks verəndə nə dəyişir

T-SQL

```
CREATE NONCLUSTERED INDEX IX_Orders_CustomerId
ON dbo.Orders(CustomerId);

SELECT Id, Total
FROM dbo.Orders
WHERE CustomerId = 7;
```

İndeks yaradılarkən mövcud uyğun məlumat oxunur, indeks qurulur və gələcək dəyişikliklərlə saxlanır. Sorğu gələndə **optimizer** mümkün planların təxmini xərcini qiymətləndirir. **Statistics**, statistika, dəyərlərin paylanması haqqında məlumatdır və neçə sətir gözlənildiyini təxmin etməyə kömək edir. İndeksin olması onun mütləq seçilməsi demək deyil.

Index seek uyğun açar və ya aralıqdan axtarışa başlayır. **Index scan** indeksin böyük hissəsini və ya hamısını gəzir. **Table scan** cədvəl məlumatını gəzir. Seek həmişə yaxşı, scan həmişə pis deyil: nəticə cədvəlin çox hissəsidirsə scan daha az iş görə bilər. Seek də geniş aralıq və çox lookup ilə bahalı ola bilər.

SELECT çox sürətlənə bilər. Amma `INSERT` yeni sətrin indeks yazısını da əlavə edir. İndekslənmiş açarın dəyişdiyi `UPDATE` indeksin uyğun hissələrini də dəyişir. `DELETE` indeks yazılarının təmizlənməsi işini yaradır. İndeks disk sahəsi tutur, aktiv səhifələri RAM-da yer tutur və dəyişikliklər transaction log yükünü artırır. Transaction log bərpa üçün dəyişiklik qeydlərini saxlayır. Yazma xərci bütün UPDATE-lərdə eyni deyil; indeksin əhatə etdiyi sütunlar və strukturdan asılıdır.

5 Composite indeksdə sıra niyə vacibdir

Composite index bir neçə sütundan ibarət açarı olan indeksdir. `(CustomerId, CreatedAt)` əvvəl müştəriyə, onun daxilində tarixə görə sıralanan telefon kitabçasına bənzəyir. `CustomerId = 7` şərti və tarix aralığı birlikdə bu quruluşa uyğundur. Yalnız tarixlə axtarışda isə ilk sütun məlum olmadığı üçün eyni səmərəli dar axtarış mümkün olmaya bilər.

T-SQL

```
CREATE INDEX IX_Orders_Customer_Date
ON dbo.Orders(CustomerId, CreatedAt DESC)
INCLUDE(Total, Status);

SELECT CreatedAt, Total, Status
FROM dbo.Orders
WHERE CustomerId = @CustomerId
  AND CreatedAt >= @From
ORDER BY CreatedAt DESC;
```

`@From` başlanğıc tarix parametridir. `DESC` azalan sıra deməkdir. **INCLUDE** sütunları indeksin yarpaq səviyyəsində nəticə üçün saxlayır; açarın axtarış sırasını genişləndirmir. **Covering index** konkret sorğunun ehtiyac duyduğu bütün məlumatı daşıyan indeksdir. “Covering” indeksin bütün sorğulara aid daimi xüsusiyyəti deyil.

Bu nümunədə CustomerId bərabərlik şərti, CreatedAt aralıq və sıralama şərtidir. “Ən çox unikal dəyəri olan sütun həmişə birinci olmalıdır” universal qayda deyil. Hədəf sorğuların bərabərlik və aralıq şərtləri, ORDER BY, paylanma və başqa sorğular birlikdə nəzərə alınır. `(Email, Status)` və `(Status, Email)` eyni bərabərliklə istifadə olunan bir sorğuda yaxın nəticə verə, başqa sorğularda fərqlənə bilər.

6 Unique filtered və aşağı cardinality

Unique index eyni açar dəyərinin təkrarına icazə vermir. Bu yalnız sürət üçün deyil, məlumat qaydasını qorumaq üçündür. **Filtered index** SQL Server-də yalnız şərtə uyğun sətirləri saxlayır. PostgreSQL-də oxşar anlayış partial index adlanır. Bu sintaksislər bütün xüsusiyyətlər üzrə tam eyni deyil.

T-SQL

```
CREATE INDEX IX_Orders_Pending
ON dbo.Orders(CreatedAt)
INCLUDE(CustomerId, Total)
WHERE Status = 'Pending';
```

Cardinality kontekstdən asılı olaraq fərqli dəyərlərin sayını və ya plan operatorunun nəticə sətir sayını bildirə bilər. **Predicate selectivity** filtrin məlumatın nə qədərini seçdiyini bildirir. 1 milyon sətrin 100-ü Pending-dirsə pay 0.01 faizdir və bu dar hissə üçün indeks faydalı ola bilər. İki mümkün status olması həmin nadir dəyəri indeksləşdirməyi avtomatik mənasız etmir.

Bool, iki qiymətli sütun, üçün indeks “həmişə faydasızdır” hökmü doğru deyil. Nadir dəyər, filtered indeks və uyğun covering sorğu fayda verə bilər. Eyni zamanda kiçik cədvəl üçün belə unique qaydanı qoruyan indeks məntiqli ola bilər. Sətir sayı üçün “1000-dən aşağı heç vaxt indeks qoyma” sərt hədd yoxdur.

7 SARGability və funksiyanın təsiri

Predicate sətirin uyğun olub olmadığını yoxlayan şərtdir. **SARGable** şərt indeksdə səmərəli axtarış aralığı yaratmağa uyğun yazılıb deməkdir. Adı Search ARGument ifadəsindən gəlir. Sütunu funksiya içində hesablamaq sadə indeksin dar seek kimi istifadəsinə mane ola bilər.

T-SQL

```
-- Tarix sutununa funksiya tetbiq edilir:
WHERE YEAR(CreatedAt) = 2026

-- Eyni ili yarimaciq interval ile secirik:
WHERE CreatedAt >= '20260101'
  AND CreatedAt <  '20270101'
```

Yarımaçıq interval başlanğıcı daxil edir, son sərhədi daxil etmir. Beləcə günün son saniyəsi və tarix dəqiqliyi problemi yaranmır. `YEAR` tarixdən ili çıxaran funksiyadır. Funksiya həmişə bütün indeks istifadəsini tam qadağan etmir; uyğun computed column indeksi və optimizer çevirmələri ola bilər. **Computed column** başqa sütunlardan hesablanan sütundur və indeks üçün ayrıca qaydalara tabedir.

`LIKE 'Ali%'` prefiks, başlanğıc hissə axtarışıdır və uyğun şəraitdə B-tree aralığına çevrilə bilər. `LIKE '%Ali%'` adi B-tree ilə eyni dar aralığı yaratmır. **Collation** mətnin sıralanma və hərf müqayisəsi qaydalarıdır. Böyük kiçik hərf, dil və operator qaydaları indeks seçiminə təsir edə bilər. İki fərqli tip müqayisə olunanda **implicit conversion**, gizli tip çevrilməsi, sütun tərəfində əlavə hesablama yaradaraq axtarışı çətinləşdirə bilər.

8 PostgreSQL indeksləri ayrıca dünyadır

PostgreSQL cədvəli heap quruluşunda saxlayır və indekslər sətir versiyalarına istinad edir. SQL Server clustered index məntiqini buraya birbaşa köçürmə. PostgreSQL-də `CLUSTER` cədvəli indeks sırasına uyğun yenidən yaza bilər, amma sonrakı yazmalar həmin sıranı avtomatik daimi saxlamır.

Tip	Məntiq və uyğun nümunə
B-tree	Sıralana bilən açarlarda bərabərlik və aralıq axtarışı. Adi standart seçimdir.
Hash	Hash kodla sadə bərabərlik axtarışı. Adi aralıq və ORDER BY üçün nəzərdə tutulmayıb.
GIN	Çox hissəli dəyərdən hissəyə geri xəritə saxlayır. JSONB, massiv və mətn tokenləri üçün uyğundur.
GiST	Müxtəlif tip və operatorlar üçün genişləndirilən axtarış çərçivəsidir. Həndəsi yaxınlıq və aralıqlar nümunədir.
SP-GiST	Məkanı hissələrə ayıran quruluşları dəstəkləyən çərçivədir. Məlumat tipinə uyğun operator sinfi tələb edir.
BRIN	Səhifə aralıqları haqqında xülasə saxlayır. Fiziki yerləşmə ilə dəyər arasında əlaqə güclüdürsə çox kiçik indekslə böyük hissələri atlaya bilər.

Hash dəyərdən hesablanan yığcam koddur; müxtəlif dəyərlər eyni koda düşə bilər və dəqiqlik yoxlaması lazım ola bilər. **GIN**, Generalized Inverted Index, məsələn “sql” etiketindən həmin etiketi daşıyan sətirlərə keçid saxlayır. **JSONB** PostgreSQL-in emala uyğun ikili JSON saxlanma tipidir. **GiST**, Generalized Search Tree, bütün operatorlara eyni sürət zəmanəti vermir. Operator class konkret tip üçün indeksin dəstəklədiyi müqayisələri müəyyən edir.

BRIN, Block Range Index, təkcə hər sətrin və ya hər səhifənin deyil, səhifə aralığının xülasəsini saxlayır. Min/max geniş yayılmış xülasə üsuludur. Namizəd aralıqlar tapılanda həqiqi sətirlər yenidən yoxlanır; buna **recheck** deyilir. Dəyərlərin fiziki sıra ilə korrelyasiyası, yəni birlikdə dəyişmə əlaqəsi, faydanı müəyyən edir. Tam ideal sıralama məcburi deyil.

PostgreSQL

```
CREATE INDEX ix_users_email_lower
ON users (lower(email));

CREATE INDEX ix_events_created_brin
ON events USING brin(created_at);

CREATE INDEX ix_posts_tags
ON posts USING gin(tags);
```

Bu alternativ nümunələr uyğun `users`, `events`, `posts` cədvəlləri və tipləri tələb edir. Birincisi expression index-dir: funksiyanın hesabladığı dəyəri indeksləşdirir. `lower` mətni kiçik hərfə çevirir. GIN üçün `tags` uyğun massiv tipi ola bilər. SQL Server-də hash indeksini adi disk rowstore B-tree-yə universal alternativ kimi yazmaq olmaz; onun memory-optimized, yaddaş üçün optimallaşdırılmış cədvəl xüsusiyyətləri ayrıca mövzudur.

9 Planı necə qiymətləndirməli

SQL Server-də Actual Execution Plan real icra barədə məlumat, `SET STATISTICS IO ON` oxunuş, `SET STATISTICS TIME ON` CPU və vaxt göstəriciləri verir. CPU prosessorun hesablamaya sərf etdiyi resursdur. **Elapsed time** istifadəçinin gözlədiyi keçən vaxtdır və bloklanma və şəbəkədən də təsirlənə bilər. Estimated plan icra etmədən təxmin göstərir.

PostgreSQL-də `EXPLAIN` planı, `EXPLAIN (ANALYZE, BUFFERS)` isə həqiqətən icra edib vaxt və buffer göstəricilərini göstərir. ANALYZE ilə yazma sorğusu da həqiqətən işləyir; onu təhlükəsiz test məlumatında yoxlamaq lazımdır. Index-only scan üçün indeksdə lazımi sütunların olması ilə yanaşı görünürlük məlumatı da təsir edir; heap-ə heç vaxt getmir demək olmaz.

Planın cost faizi real saniyə ölçüsü deyil. **Estimated rows** təxmin edilən, **actual rows** həqiqi sətir sayıdır. Böyük fərq paylanma, statistika və parametr uyğunluğu problemini göstərə bilər. Missing index tövsiyəsi də avtomatik əmr deyil; oxşar indekslər, yazma xərci və bütün iş yükü nəzərə alınmalıdır.

İndeks yoxlamasının ardıcıllığı: hədəf sorğunu və parametrləri seç, ilkin plan və oxunuşu qeyd et, bir namizəd indeks yarat, eyni şəraitdə yenidən ölç, yazma xərcini və başqa sorğuları nəzərdən keçir. Warm cache və cold cache, yəni məlumatın yaddaşda olması və olmaması, fərqli nəticə verə bilər. Laboratoriya nəticəsini bütün sistemlər üçün zəmanət kimi təqdim etmə.

10 Müsahibə məşqi

Email sütununa indeks qoyanda nə baş verir? Email-ə görə ayrıca axtarış quruluşu yaradılır. Uyğun sorğuda az səhifə oxumaq mümkün ola bilər, amma əlavə saxlanma və yazma işi yaranır. Unique seçilibsə təkrar email-lər də məhdudlaşdırılır.

Niyə indeks var amma istifadə edilmir? Nəticə çox genişdir, cədvəl kiçikdir, filtr uyğun aralıq yaratmır, statistika yanılır, tip çevrilir və ya daha ucuz başqa plan vardır. Mövcud planı görmədən “baza səhv edir” demək olmaz.

Məşq. 100 min sifarişdə `CustomerId` və tarix üzrə son 20 sifarişi seç. Əvvəl indeks olmadan, sonra tək CustomerId, sonra `(CustomerId, CreatedAt)` və uyğun INCLUDE ilə planı müqayisə et. Sıra üçün eyni tarixə qarşı `Id` əlavə açarı da düşün. Nəticəni “seek çıxdı” ilə deyil, oxunan səhifə, qayıdan sətir və yazma xərci ilə qiymətləndir.

11 Rəsmi mənbələr

- Microsoft Learn - SQL Server index types
- Microsoft Learn - Index maintenance and page density
- PostgreSQL - Index types

Yoxlama tarixi 28 oktyabr 2026. Ölçü və sürət rəqəmləri yalnız izah üçün veriləndə bunu mətn açıq bildirir. SQL nümunələri real bazada icra edilməyib.